#A Create an SSLContext for DTLS
#B Load the trusted CA certificates as a keystore
#C Initialize a TrustManagerFactory with the trusted certificates
#D Initialize the SSLContext with the trust manager

After you've created the `SSLContext`, you can use the `createEngine()` method on it to create a new `SSLEngine` object. This is the low-level protocol implementation that is normally hidden by higher-level protocol libraries like the `HttpClient` class you used in chapter 7. For a client you should pass the address and port of the server to the method when creating the engine, and configure the engine to perform the client side of the DTLS handshake by calling `setUseClientMode(true)`:

```
var address = InetAddress.getByName("localhost");
var engine = sslContext.createEngine(address, 54321);
engine.setUseClientMode(true);
```

You should then allocate buffers for sending and receiving network packets, and for holding application data. The `SSLSession` associated with an engine has methods that provide hints for the correct size of these buffers, which you can query to ensure you allocate enough space, as shown in the following code:

```
var session = engine.getSession();     #A
var recvBuf =     #B
    ByteBuffer.allocate(session.getPacketBufferSize());    #B
var sendBuf =     #B
    ByteBuffer.allocate(session.getPacketBufferSize());    #B
var appData =     #B
```